

Technische Hochschule Deggendorf

Fakultät Angewandte Informatik

Studiengang Bachelor Cyber-Security

Kurs: Security Engineering – SS 2026

Projektdokumentation

Secure DevOps (DevSecOps) – Sicherheitstechnische Betrachtung automatisierter CI/CD-Pipelines

Vorgelegt von:

Christof Renner (22301943)
Manuel Friedl (12306626)

Betreuer:

Prof. Dr. Martin Schramm

Datum: 25. April 2026

Inhaltsverzeichnis

1	Einleitung	3
1.1	Motivation und Problemstellung	3
1.2	Zielsetzung	3
1.3	Abgrenzung	3
1.4	Aufbau der Arbeit	3
2	Grundlagen	4
2.1	DevSecOps	4
2.2	Software Supply Chain Security	4
2.3	Frameworks	4
2.4	Security Engineering Framework	4
3	System unter Betrachtung	5
3.1	Architekturüberblick	5
3.2	Schützenswerte Assets	5
3.3	Trust Boundaries	6
3.4	Angreifermodell	6
4	Bedrohungsanalyse (STRIDE)	6
5	Gegenmaßnahmen und Implementierung	7
5.1	Source-Stufe	7
5.2	Build-Stufe	8
5.3	Test- und Scan-Stufe	8
5.4	Package- und Sign-Stufe	8
5.5	Deploy-Stufe	8
5.6	Auszug aus der CI-Implementierung	9
5.7	Mapping Bedrohung → Maßnahme	9
5.8	Fallstudie: tj-actions/changed-files (März 2025)	10
6	Evaluation	10
6.1	SLSA-Einstufung	10
6.2	OpenSSF Scorecard	11
6.3	Coverage der STRIDE-Kategorien	11
6.4	Validierung gegen OWASP Top 10 CI/CD	11
6.5	Restrisiken	12
6.6	Zielkonflikt Geschwindigkeit vs. Assurance	12
6.7	Faktor Mensch und Anreize	12
7	Fazit und Ausblick	13

1 Einleitung

1.1 Motivation und Problemstellung

Moderne Softwareentwicklung basiert auf hochgradig automatisierten Build-, Test- und Deployment-Prozessen. CI/CD-Pipelines sind tief in cloud-native Infrastrukturen integriert und verfügen über weitreichende Zugriffsrechte auf sensible Ressourcen wie Quellcode-Repositories, Secrets, Container-Registries und Produktionsumgebungen. Angriffe auf Softwarelieferketten – etwa SolarWinds (2020), Codecov (2021), Log4Shell (2021) und der 3CX-Vorfall (2023) – belegen, dass der Entwicklungsprozess selbst zu einem primären Angriffsziel geworden ist.

Aus Sicht des Security Engineering ist eine Pipeline weniger ein Tooling-Problem als ein *Systemproblem*: Sie verbindet menschliche Identitäten, externe Abhängigkeiten, Build-Hosts, Registries und Deployment-Targets über mehrere Vertrauensgrenzen hinweg. Eine isolierte Betrachtung einzelner Werkzeuge greift daher zu kurz.

1.2 Zielsetzung

Ziel der Arbeit ist eine systematische sicherheitstechnische Betrachtung einer repräsentativen CI/CD-Pipeline. Konkret werden:

- schützenswerte Assets und Trust Boundaries identifiziert,
- Bedrohungen je Pipeline-Stufe nach STRIDE strukturiert analysiert,
- geeignete Sicherheitsmechanismen abgeleitet und in einer prototypischen Pipeline (GitHub Actions) umgesetzt,
- die Wirksamkeit dieser Maßnahmen anhand von SLSA-Levels und OpenSSF-Scorecard evaluiert,
- verbleibende Restrisiken transparent gemacht.

1.3 Abgrenzung

Nicht Gegenstand der Arbeit sind: vollständige Härtung des Cloud-Tenants, Penetrationstests der Pipeline-Plattform selbst sowie organisatorische Maßnahmen jenseits der Branch-Protection-Konfiguration. Die Pipeline wird auf GitHub Actions umgesetzt; die Übertragbarkeit auf GitLab CI wird qualitativ diskutiert.

1.4 Aufbau der Arbeit

Kapitel 2 klärt die Begriffe DevSecOps, Supply-Chain-Security und die zugrundeliegenden Frameworks (STRIDE, SLSA, SSDF). Kapitel 3 beschreibt das modellierte System und seine Trust Boundaries. Kapitel 4 enthält das STRIDE-Threat-Model. Kapitel 5 ordnet jeder Bedrohung konkrete Gegenmaßnahmen zu und dokumentiert die Implementierung

im Prototyp. Kapitel 6 bewertet die Wirksamkeit, Kapitel 7 fasst zusammen und diskutiert Restrisiken.

2 Grundlagen

2.1 DevSecOps

DevSecOps bezeichnet die Integration sicherheitstechnischer Aktivitäten in den DevOps-Lebenszyklus mit dem Ziel, Sicherheit als kontinuierlichen, automatisierten Bestandteil von Build und Deployment zu etablieren („shift left“ und „shift right“). Sicherheitsmaßnahmen werden nicht nachgelagert aufgesetzt, sondern als Teil derselben Pipeline ausgeführt, die auch das Artefakt produziert.

2.2 Software Supply Chain Security

Eine Software Supply Chain umfasst alle Personen, Prozesse, Werkzeuge und Abhängigkeiten, die zur Erstellung und Auslieferung eines Artefakts beitragen. Angriffsklassen nach [1] sind insbesondere:

- **Source**-Angriffe (z. B. kompromittierte Maintainer-Accounts, Typosquatting),
- **Build**-Angriffe (z. B. kompromittierte Runner, manipulierte Compiler/Plugins),
- **Dependency**-Angriffe (z. B. Übernahme verwaister Pakete),
- **Distribution**-Angriffe (z. B. Image-Tag-Replacement in Registries).

2.3 Frameworks

STRIDE

Microsoft-Methodik zur Bedrohungsmodellierung mit den Kategorien *Spoofing*, *Tampering*, *Repudiation*, *Information Disclosure*, *Denial of Service*, *Elevation of Privilege*.

SLSA v1.0

Supply-chain Levels for Software Artifacts definiert Anforderungen an Build-Integrität in vier Stufen (Build L1–L3, Source L1–L3).

NIST SSDF (SP 800-218)

Secure Software Development Framework – Praktiken über den gesamten SDLC.

OpenSSF Scorecard

Automatisierte Bewertung von Repository-Posture anhand objektiver Checks.

2.4 Security Engineering Framework

Die Arbeit folgt dem in der Lehrveranstaltung eingeführten Vier-Quadranten-Modell des Security Engineering nach Anderson: *Policy*, *Mechanism*, *Assurance* und *Incentives*. Ta-

belle 1 ordnet jedem Quadranten konkrete Bestandteile der untersuchten Pipeline zu; auf den Quadranten *Incentives* wird in Abschnitt 6.7 vertiefend eingegangen.

Quadrant	Umsetzung in der Pipeline
Policy	Branch-Protection-Regeln, CODEOWNERS, Required Status Checks, Coverage-Schwelle 80 %, HIGH/CRITICAL als Build-Breaker, Manual-Approval-Gate für Production.
Mechanism	Bandit, CodeQL, pip-audit, Trivy, Syft (SBOM), Cosign Keyless Signing, SLSA-Provenance, OIDC-Federation, Action-Pinning auf Commit-SHA.
Assurance	SLSA-Level-Bewertung, OpenSSF Scorecard (wöchentlich), SARIF-Upload in Code-Scanning, Cosign-Verify im CD-Workflow, Sigstore Rekord Transparency Log.
Incentives	Vier-Augen-Prinzip via CODEOWNERS, automatisierte Dependabot-PRs reduzieren Reibung für sichere Updates, parallele Jobs + <code>cancel-in-progress</code> mildern den Geschwindigkeits-Konflikt (siehe Abschnitt 6.7).

Tabelle 1: Mapping der Pipeline-Bestandteile auf das Security-Engineering-Framework.

3 System unter Betrachtung

3.1 Architekturüberblick

Untersucht wird eine prototypische, container-basierte Pipeline für eine Python-Webanwendung. Die wesentlichen Stufen sind in Abbildung 1 dargestellt.

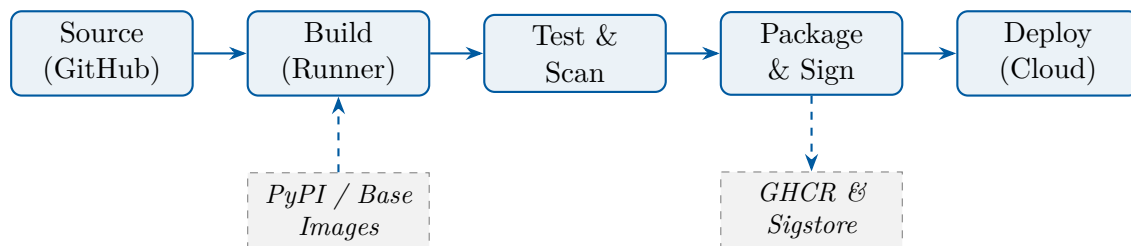


Abbildung 1: Pipeline-Stufen und externe Vertrauensanker.

3.2 Schützenswerte Assets

Asset	Schutzziel (CIA + AAA)
Quellcode (main)	Integrität, Authentizität
Build-Artefakt (Image)	Integrität, Authentizität, Verfügbarkeit
Secrets (Cloud-Cred., Tokens)	Vertraulichkeit, Integrität
SBOM & Provenance	Integrität, Nicht-Abstreitbarkeit

Asset	Schutzziel (CIA + AAA)
Pipeline-Konfiguration	Integrität, Autorisierung
Build-Runner-Identität (OIDC)	Authentizität, Vertraulichkeit

3.3 Trust Boundaries

1. Entwickler-Workstation ↔ GitHub
2. GitHub-Plattform ↔ Build-Runner
3. Build-Runner ↔ Paket-/Image-Registries (PyPI, Docker Hub, GHCR)
4. Build-Runner ↔ Sigstore (Fulcio/Rekor)
5. Pipeline ↔ Deployment-Target (Cloud-API)

3.4 Angreifermodell

Drei relevante Profile werden betrachtet:

A1 – Externer Angreifer

kein Repo-Zugriff; nutzt Phishing, Typosquatting, Schwachstellen in Abhängigkeiten.

A2 – Kompromittierter Mitwirkender

gültige, aber begrenzte Schreibrechte (z. B. Fork-PR, einzelner Maintainer).

A3 – Insider/Plattform

Admin-Rechte am Repo bzw. kompromittierter Self-hosted-Runner.

4 Bedrohungsanalyse (STRIDE)

Die folgende Tabelle dokumentiert pro Pipeline-Stufe die wesentlichen Bedrohungen, klassifiziert nach STRIDE. Die Risiko-Einstufung folgt einer qualitativen Skala (niedrig/mittel/hoch) auf Basis von Eintrittswahrscheinlichkeit und Schadensausmaß im Kontext einer Studienarbeitstypischen Demo-Pipeline.

Stufe	STRIDE	Bedrohung	Angr.	Risiko
Source	S	Push mit gestohlenen Maintainer-Credentials	A1/A2	hoch
Source	T	Manipulation von Pipeline-Definitionen via PR	A2	hoch
Source	R	Unsignierte Commits, fehlende Audit-Trails	A2	mittel
Source	I	Secrets im Klartext im Repo (Logs, Con-figs)	A1	hoch

Stufe	STRIDE	Bedrohung	Angr.	Risiko
Build	T	Bösartige transitive Dependency (Confused-Deps, Typosquat)	A1	hoch
Build	T	Manipulation des Base-Images (Tag-Replacement)	A1	hoch
Build	E	Workflow mit zu weiten permissions (RW everywhere)	A2	hoch
Build	E	pull_request_target mit secrets ausgesetzt	A1	hoch
Build	I	Cache-Poisoning zwischen PRs	A2	mittel
Test	D	Lange laufende Scans blockieren Deployments	A1	niedrig
Test	T	Test-Bypass via geänderte Workflow-Bedingungen	A2	mittel
Pkg	T	Ungesignedes Image, Tag-Hijack in Registry	A1	hoch
Pkg	R	Fehlende Provenance / SBOM	–	mittel
Deploy	S	Long-lived Cloud-Credentials in Repo-Secrets	A2	hoch
Deploy	E	Fehlende Approval-Gates für Production	A2	hoch
Deploy	I	Verbose Logs leaken Tokens / PII	A1	mittel

Tabelle 3: STRIDE-Mapping der untersuchten Pipeline.

5 Gegenmaßnahmen und Implementierung

5.1 Source-Stufe

Branch Protection & CODEOWNERS

main ist geschützt; Merges erfordern ein Review durch Code-Owner (Vier-Augen-Prinzip), bestandene Pflicht-Checks, signierte Commits und lineare History.

Signierte Commits

Entwickler signieren mit SSH/GPG; nicht-signierte Commits werden serverseitig abgelehnt.

Secret-Scanning

gitleaks läuft als Pre-Commit-Hook und in der CI; GitHub Push-Protection ergänzt das.

Pinned Actions

Alle GitHub-Actions sind auf Commit-SHA gepinnt, um Tag-Retag-Angriffe (z. B. tj-actions/changed-files 03/2025) zu verhindern.

5.2 Build-Stufe

Least-Privilege-Permissions

Workflow-Default ist `permissions: {}`; jeder Job opt-in mit minimalen Scopes.

Ephemere Runner

Nutzung von GitHub-hosted Runnern (frische VM pro Job); Self-hosted bewusst nicht verwendet.

Reproduzierbares Image

Multi-stage `Dockerfile` mit gepinntem Base-Image, Non-Root-User (UID 10001), Read-only-Filesystem-tauglich.

Dependency-Pinning

`requirements.txt` mit exakten Versionen; Updates über Dependabot-PRs, die die volle Pipeline durchlaufen.

5.3 Test- und Scan-Stufe

- **SAST:** Bandit (Python) + GitHub CodeQL, Ergebnisse als SARIF in die GitHub Code-Scanning-API.
- **SCA:** `pip-audit` gegen die OSV-Datenbank, `-strict` bricht den Build bei bekannter CVE ab.
- **Container-Scan:** Trivy auf das gebaute Image, `HIGH,CRITICAL` mit `exit-code: 1`.
- **Coverage-Gate:** `pytest -cov-fail-under=80`.

5.4 Package- und Sign-Stufe

SBOM

Syft erzeugt CycloneDX-JSON aus dem Image; das SBOM wird als Attestation an das Image angeheftet.

Cosign Keyless Signing

Signatur via Sigstore Fulcio mit dem OIDC-Token des Workflows; keine langlebigen Schlüssel im Repo.

SLSA-Provenance

`actions/attest-build-provenance` erzeugt eine verifizierbare Provenance-Attestation (SLSA v1).

5.5 Deploy-Stufe

OIDC-Federation

Cloud-Logins (Azure / AWS) erfolgen via OIDC – keine statischen Cloud-Credentials in Repo-Secrets.

Environments mit Approval

production ist GitHub-Environment mit Pflicht-Reviewer und Wait-Timer.

Cosign-Verify vor Deploy

Vor dem Rollout verifiziert der CD-Workflow Signatur *und* SLSA-Provenance des Image-Digests.

Image-Pinning per Digest

Deployments referenzieren @sha256:..., nicht Tags.

5.6 Auszug aus der CI-Implementierung

Listing 1 zeigt die Job-Topologie und die zentralen Sicherheits-Gates.

```
1 permissions: {}                                # default-deny
2
3 jobs:
4   lint-test:
5     permissions: { contents: read }
6     steps:
7       - uses: actions/checkout@11bd71901bbe5b1630ceea73d27597364c9af683
8         # v4.2.2
9       with: { persist-credentials: false }
10  build-image:
11    needs: [lint-test, sast, sca, secret-scan]
12    permissions:
13      contents: read
14      packages: write
15      id-token: write          # OIDC fuer cosign keyless
16      attestations: write
```

Listing 1: Auszug .github/workflows/ci.yml: Permissions & Pin auf SHA

```
1 cosign verify \
2   --certificate-identity-regexp "https://github.com/$REPO/.github/
3   workflows/.+" \
4   --certificate-oidc-issuer "https://token.actions.githubusercontent.com" \
5   "$IMAGE"
6
7 cosign verify-attestation --type slsaprovenance \
8   --certificate-identity-regexp "https://github.com/$REPO/.github/
9   workflows/.+" \
10  --certificate-oidc-issuer "https://token.actions.githubusercontent.com" \
11  "$IMAGE"
```

Listing 2: Verifikation vor Deploy (cd.yml)

5.7 Mapping Bedrohung → Maßnahme

Bedrohung (Tab. 3)	Maßnahme
Maintainer-Credential-Diebstahl	MFA, signierte Commits, CODEOWNERS-Review
Manipulation Pipeline-Definition	CODEOWNERS auf <code>.github/</code> , Required Checks
Bösartige Dependency	pip-audit, Dependabot, Pinning, SBOM-Diff
Base-Image-Replacement	Pinning + Trivy + Image-Digest in Deploy
Zu weite <code>permissions</code>	default-deny + per-Job opt-in
<code>pull_request_target</code> Leak	Trigger nicht verwendet; PRs ohne Secrets
Tag-Hijack in Registry	Cosign-Verify + Digest-Pinning beim Deploy
Fehlende Provenance	SLSA-Provenance via <code>attest-build-provenance</code>
Long-lived Cloud-Creds	OIDC-Federation, keine statischen Secrets
Fehlendes Approval Production	GitHub Environment Reviewer + Wait-Timer

Tabelle 4: Bedrohungs-Maßnahmen-Matrix.

5.8 Fallstudie: `tj-actions/changed-files` (März 2025)

Die Action `tj-actions/changed-files` wurde im März 2025 kompromittiert: Angreifer überschrieben mehrere veröffentlichte Tags so, dass sie auf einen manipulierten Commit zeigten, der Secrets aus dem Runner-Memory in die Build-Logs exfiltrierte. Workflows, die die Action ohne SHA-Pinning eingebunden hatten (`tj-actions/changed-files@v44`), zogen automatisch die manipulierte Version. Über zehntausend Repositories waren betroffen, in mehreren Fällen wurden GitHub-PATs und Cloud-Credentials öffentlich auf GitHub-Logs sichtbar.

Der Vorfall illustriert exakt die Bedrohungsklasse *Source/Build-T* aus Tabelle 3. Die in dieser Arbeit umgesetzte Maßnahme – durchgängiges Pinning aller Actions auf den Commit-SHA mit einem nachgestellten Versions-Kommentar – hätte den Angriff vollständig neutralisiert: Selbst wenn der Tag `v44` auf einen böartigen Commit umgehängt wird, verweist der Pin `tj-actions/changed-files@<sha> # v44` weiterhin auf die auditierte Version. Sigstore/Cosign hätte die Auswirkung zusätzlich begrenzt, da das resultierende Artefakt keine gültige Signatur unter der OIDC-Identität des Workflows getragen hätte.

6 Evaluation

6.1 SLSA-Einstufung

Die umgesetzte Pipeline erreicht nach [1]:

- **Build L3:** gehosteter, ephemerer Builder; Provenance ist nicht fälschbar (Sigstore-Signatur via OIDC); Build-Skript ist versioniert.
- **Source L2:** versioniert, retained history, Two-person review via CODEOWNERS.

6.2 OpenSSF Scorecard

Der Scorecard-Workflow läuft wöchentlich. Erwartete Scores $\geq 8/10$ für *Branch-Protection*, *Pinned-Dependencies*, *Token-Permissions*, *Signed-Releases*, *SAST*, *Vulnerabilities*.

6.3 Coverage der STRIDE-Kategorien

Tabelle 5 fasst zusammen, welche STRIDE-Kategorien durch die Maßnahmen mitigiert sind.

Kategorie	Mitigiert	Wesentliche Maßnahme
Spoofing	vollständig	OIDC, signierte Commits, Cosign
Tampering	weitgehend	Pinning, Provenance, Verify-Gate
Repudiation	vollständig	Provenance + Rekord-Transparency-Log
Information Disclosure	weitgehend	gitleaks, least-privilege Tokens
Denial of Service	teilweise	concurrency-cancel, Cache-Strategie
Elevation of Privilege	vollständig	default-deny permissions, Environments

Tabelle 5: STRIDE-Coverage durch implementierte Maßnahmen.

6.4 Validierung gegen OWASP Top 10 CI/CD

Zur Plausibilisierung der STRIDE-Analyse wird die abgeleitete Maßnahmenliste zusätzlich gegen die OWASP-Top-10-CI/CD-Risiken [6] geprüft. Tabelle 6 zeigt die Abdeckung.

Nr.	OWASP-Risiko	Wesentliche Maßnahme
1	CICD- Insufficient Flow Control Mechanisms	Branch-Protection, CODEOWNERS, Environments mit Approval
2	CICD- Inadequate Identity & Access Mgmt	OIDC-Federation, least-privilege permissions
3	CICD- Dependency Chain Abuse	Pinning, pip-audit, Dependabot, SBOM-Diff
4	CICD- Poisoned Pipeline Execution (PPE)	Kein <code>pull_request_target</code> , CODEOWNERS auf <code>.github/</code>
5	CICD- Insufficient PBAC	Per-Job permissions, Environment-Reviewer
6	CICD- Insufficient Credential Hygiene	gitleaks, Push-Protection, Cosign keyless (keine Long-lived Keys)
7	CICD- Insecure System Configuration	Scorecard wöchentlich, Action-Pinning, default-deny Permissions
8	CICD- Ungoverned Usage of 3rd-Party Services	Kuratierte Action-Liste, SHA-Pin, Renovate/Dependabot Audit
9	CICD- Improper Artifact Integrity Validation	Cosign-Verify + SLSA-Provenance vor Deploy, Digest-Pinning

Nr.	OWASP-Risiko	Wesentliche Maßnahme
CICD-10	Insufficient Logging & Visibility	SARIF-Upload, Rekorder, Transparency-Log, Audit-Logs (org-level)

Tabelle 6: Abdeckung der OWASP-Top-10-CI/CD-Risiken durch die implementierte Pipeline.

Alle zehn Risikoklassen werden adressiert. CICD-10 ist organisatorisch abhängig (Audit-Logs auf Org-Ebene erfordern entsprechende GitHub-Enterprise-Konfiguration) und damit nur teilweise innerhalb der Pipeline selbst mitigierbar.

6.5 Restrisiken

- **Plattform-Kompromittierung** (GitHub, Sigstore): nicht durch die Pipeline mitigierbar; Vertrauensanker.
- **Insider mit Admin-Rechten** kann Branch-Protection deaktivieren. Mitigation: Audit-Logging, Org-Policies.
- **Zero-Day in transitiver Abhängigkeit** – pip-audit erkennt nur bekannte CVEs.
- **Secret-Exfiltration via Build-Logs** bei fahrlässigem `echo`; nur durch Code-Review und `add-mask` mitigierbar.

6.6 Zielkonflikt Geschwindigkeit vs. Assurance

Strikte Gates (HIGH/CRITICAL fail, Coverage 80 %, manuelle Approvals) verlangsamen Deployments. Die Pipeline mildert dies durch parallele Jobs, GHA-Caches und `concurrency: cancel-in-progress` für PRs. Production behält bewusst eine Approval-Hürde – Geschwindigkeit darf hier nicht über Assurance dominieren.

6.7 Faktor Mensch und Anreize

Technische Mechanismen entfalten ihre Wirkung nur, wenn die beteiligten Akteure – Entwickler, Reviewer, Plattform-Administratoren – sie auch konsequent anwenden. Drei Spannungsfelder sind in der Praxis besonders relevant.

Geschwindigkeit vs. Reibung. Sicherheits-Gates wie Coverage-Schwellen, signierte Commits oder Manual-Approval-Schritte verlangsamen den Feedback-Zyklus. Wird die Reibung als unverhältnismäßig empfunden, neigen Teams dazu, Schutzmaßnahmen zu umgehen – etwa durch `[skip ci]`-Commits, `-no-verify` bei pre-commit-Hooks oder das pauschale Akzeptieren von HIGH-Findings als „false positive“. Die Pipeline mildert dies, indem sie schnelle Jobs (Lint, Tests) parallel zu langsamen (CodeQL, Trivy) ausführt und PRs bei neuen Commits via `cancel-in-progress` abbricht.

Ownership-Diffusion. „Security ist Aufgabe des Security-Teams“ ist ein verbreitetes Anti-Pattern. CODEOWNERS verschiebt das Eigentum für sicherheitskritische Pfade (`.github/`, `Dockerfile`) explizit auf die Pipeline-Verantwortlichen und macht Reviews damit zum unumgehbaren Default. Die OpenSSF-Scorecard liefert wiederum eine *externe* Bewertung, die Diskussionen über Posture-Defizite versachlicht.

Alarmmüdigkeit. Eine Pipeline, die bei jedem Push fünfzig Findings produziert, wird ignoriert. Die Implementierung adressiert dies durch die Trennung in Build-Breaker (`HIGH`, `CRITICAL`, mit `ignore-unfixed: true`) und reine Reportings via SARIF. Damit bleibt jeder Build-Abbruch handlungsrelevant, und Findings ohne unmittelbaren Fix verschwinden nicht aus der Sicht (Code-Scanning-View), erzwingen aber keinen Stillstand.

Anreiz für Maintainer. Dependabot erzeugt Updates als kleine, isolierte PRs, die die volle Pipeline durchlaufen. Damit wird das Aktualisieren von Abhängigkeiten zur *billigsten* statt zur teuersten Option – ein klassischer Anreiz-Hebel im Sinne des Security-Engineering-Frameworks (Tab. 1).

7 Fazit und Ausblick

Die Arbeit zeigt, dass sich eine moderne CI/CD-Pipeline mit zugänglichen Mitteln (GitHub Actions, Sigstore, Trivy, Syft) systematisch entlang einer STRIDE-Analyse absichern lässt und dabei nachweisbare Eigenschaften gemäß SLSA erreicht. Wesentlich ist nicht das einzelne Tool, sondern die Kombination: *least-privilege Identitäten, gepinnte Abhängigkeiten, signierte Artefakte mit Provenance, verifizierte Deployments*.

Weiterführende Arbeiten könnten:

- die Übertragbarkeit der Maßnahmen auf GitLab CI quantitativ vergleichen,
- Policy-as-Code (z. B. Kyverno, Sigstore Policy Controller im Cluster) als zusätzliche Verifikationsschicht ergänzen,
- formale Analysen zu Pipeline-Konfigurationen (z. B. `zizmor`, `actionlint`) mit Beweisbarkeit der *permissions*-Minimalität verbinden.

Literatur

- [1] OpenSSF, *Supply-chain Levels for Software Artifacts (SLSA) v1.0*, <https://slsa.dev/spec/v1.0/>, abgerufen 2026.
- [2] NIST, *Secure Software Development Framework (SSDF) Version 1.1*, SP 800-218, 2022.
- [3] Microsoft, *The STRIDE Threat Model*, Microsoft Learn, abgerufen 2026.
- [4] OpenSSF, *Scorecard*, <https://github.com/ossf/scorecard>, abgerufen 2026.

- [5] Sigstore Project, *Cosign / Fulcio / Rekor*, <https://www.sigstore.dev/>, abgerufen 2026.
- [6] OWASP, *Top 10 CI/CD Security Risks*, <https://owasp.org/www-project-top-10-ci-cd-security-risks/>, abgerufen 2026.